
IUT A - Université de Lille
Année scolaire 2025 - 2026
Département informatique

**Stage de mise en place d'une démarche DevSecOps pour le cycle
de vie d'une application web dans le Cloud
Entreprise Devox à Rabat (Maroc)**

Rapport de stage



Soutenu le Lundi 31 Août 2026
Par KOUDAMRA Wassim

Encadrant entreprise : MOURTADA Raouf
Encadrant universitaire : PETER Yvan

BUT 3 Informatique
Parcours Déploiement d'Applications Communicantes et Sécurisées

Remerciements :

Je souhaiterais tout d'abord remercier M. le Colonel Intendant Militaire Younes ARAB ainsi que le Colonel Khalid OUALLA, Attaché Militaire Naval et de l'Air auprès de l'ambassade du Maroc à Paris, sans qui la mise en contact avec l'entreprise d'accueil DEVOX n'aurait pas été possible.

Je remercie l'IUT de l'Université de Lille ainsi que toute l'équipe pédagogique pour leur accompagnement, ainsi que leur pédagogie au cours de cette troisième année au sein du BUT Informatique Parcours Déploiement d'Applications Communicantes et Sécurisées.

Je tiens tout particulièrement à remercier et à témoigner toute ma reconnaissance aux personnes suivantes, pour l'expérience enrichissante et pleine d'intérêt qu'elles m'ont fait vivre durant ces 3 mois au sein de l'entreprise DEVOX

M. Ait Youssef SOUHAIL, Directeur de la société DEVOX pour son accueil et la confiance qu'il m'a accordé au sein de sa structure.

M. Raouf MOURTADA, RSSI et DevOps au sein de la société DEVOX et mon tuteur lors de ce stage et dont l'accompagnement m'a été précieux tout au long de cette expérience.

Mme. Khaoula HAMDOUNI, secrétaire au sein de la société DEVOX pour son accompagnement du côté administratif de ce stage.

Encore merci à toute l'équipe de l'entreprise DEVOX pour leur accueil, leur bienveillance et leur accompagnement tout au long de ce stage.

Sommaire :

Résumé du rapport en français et en anglais :

0) Introduction :

I) L'entreprise Devox :

II) Sujet du stage et présentation du projet :

III) Présentation des outils et supports :

IV) Conclusion :

V) Lexique :

VI) Source, annexes et bibliographie :

Résumé du rapport de stage en français :

Dans le cadre de ma troisième année de Bachelor Universitaire de Technologie (BUT) Informatique, parcours Déploiement d'Applications Communicantes et Sécurisées, à l'IUT de Lille, j'ai effectué un stage de 13 semaines à partir de juin 2026 au sein de l'entreprise DEVOX, basée à Rabat au Maroc. DEVOX est une société marocaine créée en 2014, spécialisée dans le développement web et mobile, la gestion d'infrastructures cloud et le conseil IT, et travaillant aussi bien avec des clients du secteur public (Gendarmerie Royale, Ministère des Finances, universités) que du secteur privé (Redal, Orsys, Injaze...).

Ce stage s'inscrit dans une démarche DevSecOps appliquée à la sécurisation du cycle de vie d'une application web destinée à être hébergée dans le cloud. Après avoir eu le choix entre déployer une application existante ou en concevoir une nouvelle, j'ai choisi de créer de zéro une solution de monitoring pour serveur Ubuntu : **Sentinelle**. L'objectif était de proposer un outil de supervision et de sécurisation fonctionnant en local, sans dépendre d'outils tiers, afin de limiter la surface d'exposition aux cyberattaques.

Le projet a été développé de manière itérative, en quatre versions. La version 1 reposait sur une dizaine de scripts Bash lancés manuellement. La version 2 a automatisé cette collecte en arrière-plan via Systemd, avec stockage des données dans MySQL et un premier mécanisme de blocage des IP malveillantes inspiré de Fail2Ban. La version 3 a introduit un dashboard web affichant en temps réel la consommation CPU, RAM et disque. La version 4, version finale, a ajouté un espace administrateur sécurisé par authentification JWT et double authentification (2FA/TOTP), un journal détaillé des événements de sécurité, ainsi qu'un suivi de l'espace SWAP.

Après une phase de tests en local sous Kubuntu 26.04, la solution a été déployée sur une machine virtuelle Ubuntu Server à l'aide d'un script maître automatisant l'installation complète (dépendances, base de données, backend Spring Boot, frontend, configuration Apache en reverse proxy, services Systemd, permissions POSIX). L'ensemble du projet a été versionné sur GitHub, puis conteneurisé avec Docker et Docker Compose en quatre microservices (collecteur, backend, interface web, base de données), garantissant sa portabilité indépendamment de tout fournisseur cloud.

Sur le plan technique, la conception de Sentinelle s'est appuyée sur l'étude comparative de trois solutions de référence : Fail2Ban pour la détection et le blocage des IP agressives, Icinga pour la logique de supervision par sondes et seuils d'état, et Cacti pour l'archivage et la restitution graphique des métriques. Le choix de MySQL comme SGBD, plutôt que MongoDB, a été justifié par la nature relationnelle et structurée des données à stocker (hôtes, métriques, services, événements, utilisateurs), avec un schéma normalisé en cinq tables et plusieurs mesures de durcissement (moteur InnoDB, suppression en cascade, hachage BCrypt, RBAC, écoute restreinte à l'interface locale).

Ce stage m'a permis de conduire un projet dans son intégralité, de la conception à la sécurisation et au déploiement conteneurisé, en renforçant mes compétences en administration système Linux, en orchestration Docker et en sécurité applicative. Comparée à mon stage de BTS CIEL, où les choix d'architecture étaient déjà arrêtés, cette expérience chez DEVOX m'a demandé une posture d'ingénierie globale et une réelle autonomie. Elle confirme mon orientation vers les métiers de la cybersécurité et constitue, pour DEVOX, un socle interne souverain et évolutif pour la supervision et la sécurisation de ses serveurs.

Summary of the internship report in English :

As part of my third year of the Bachelor's Degree in Technology (BUT) in Computer Science, specialising in the Deployment of Communicating and Secure Applications, at the IUT in Lille, I undertook a 13-week work placement starting in June 2026 with DEVOX, a company based in Rabat, Morocco. DEVOX is a Moroccan company founded in 2014, specialising in web and mobile development, cloud infrastructure management and IT consultancy, and working with clients in both the public sector (Royal Gendarmerie, Ministry of Finance, universities) and the private sector (Redal, Orsys, Injaze, etc.).

This placement forms part of a DevSecOps approach applied to securing the lifecycle of a web application intended to be hosted in the cloud. After being given the choice between deploying an existing application or designing a new one, I chose to build a monitoring solution for Ubuntu servers from scratch: Sentinelle. The aim was to provide a monitoring and security tool that runs locally, without relying on third-party tools, in order to minimise the attack surface.

The project was developed iteratively, across four versions. Version 1 relied on around ten Bash scripts run manually. Version 2 automated this data collection in the background via Systemd, with data stored in MySQL and an initial mechanism for blocking malicious IP addresses inspired by Fail2Ban. Version 3 introduced a web dashboard displaying CPU, RAM and disk usage in real time. Version 4, the final version, added an admin area secured by JWT authentication and two-factor authentication (2FA/TOTP), a detailed security events log, and SWAP space monitoring.

Following a local testing phase on Kubuntu 26.04, the solution was deployed on an Ubuntu Server virtual machine using a master script that automated the entire installation (dependencies, database, Spring Boot backend, frontend, Apache reverse proxy configuration, Systemd services, POSIX permissions). The entire project was versioned on GitHub, then containerised using Docker and Docker Compose into four microservices (collector, backend, web interface, database), ensuring its portability regardless of any cloud provider.

From a technical perspective, the design of Sentinelle was based on a comparative study of three benchmark solutions: Fail2Ban for detecting and blocking malicious IP addresses, Icinga for monitoring logic using probes and status thresholds, and Cacti for archiving and graphically displaying metrics. The choice of MySQL as the DBMS, rather than MongoDB, was justified by the relational and structured nature of the data to be stored (hosts, metrics, services, events, users), with a normalised schema comprising five tables and several hardening measures (InnoDB engine, cascade delete, BCrypt hashing, RBAC, and listening restricted to the local interface).

This work placement enabled me to lead a project from start to finish, from design through to security and containerised deployment, whilst strengthening my skills in Linux system administration, Docker orchestration and application security. Compared to my BTS CIEL placement, where the architectural choices had already been finalized, this experience at DEVOX required me to adopt a holistic engineering approach and work with genuine autonomy. It confirms my career path towards cybersecurity and provides DEVOX with a robust, scalable in-house foundation for monitoring and securing its servers.

0) Introduction :

Dans le cadre de ma troisième année de Bachelor Universitaire de Technologie (BUT) en Informatique parcours Déploiement d'Applications Communicantes et Sécurisées à l'IUT de Lille, j'ai effectué un stage d'une durée de 13 semaines à compter de juin 2026 au sein de la société DEVOX, basée à Rabat au Maroc. Ce stage de fin d'études constitue une étape décisive de ma formation, me permettant d'appliquer sur le terrain l'ensemble des compétences acquises en matière de développement logiciel, d'administration système, de réseaux et de sécurisation des infrastructures.

Mon accueil a été assuré par DEVOX, une société marocaine créée en 2014, spécialisée dans les Technologies de l'Information, le développement web et mobile, la gestion d'infrastructures cloud et le conseil IT. Basée à Rabat, DEVOX se constitue d'une équipe d'une quinzaine de membres passionnés et chevronnés, diplômés des grandes écoles d'ingénieurs. Elle possède un savoir-faire et une expertise reconnus dans la conception, le développement, la mise en œuvre et la gestion de solutions IT avancées, particulièrement axées sur les applications B2C (Business to Consumer) facilitant la mise en relation directe entre une entreprise et un consommateur final. L'entreprise est engagée à accompagner ses clients, issus du secteur public comme du secteur privé, dans la digitalisation de leurs processus métiers. Pour DEVOX, ce projet constitue une base technique interne autonome, réutilisable et évolutive pour la supervision et la sécurisation automatisées de ses serveurs Ubuntu sans dépendre de solutions externes complexes.

Ce stage s'inscrit dans la mise en place d'une démarche DevSecOps appliquée à la sécurisation du cycle de vie des applications web, en préparant leur hébergement dans des environnements cloud. Concernant le thème de la solution à déployer, l'équipe technique de DEVOX m'a accordé une liberté de choix entre deux options : déployer sur un serveur Ubuntu une application déjà existante développée par les ingénieurs de DEVOX, ou concevoir une solution de zéro et la déployer par la suite. Afin de mettre davantage en pratique mes connaissances et d'améliorer mes compétences sur l'ensemble du cycle de vie d'un logiciel, mon choix s'est naturellement porté vers la seconde option avec la création de la solution Sentinelle.

L'idée générale de cet outil est de proposer une solution de sécurisation et de supervision des événements d'un serveur Ubuntu en limitant sa dépendance à des outils tiers. Dans le cadre de ce projet, la limitation du recours à des outils tiers permet notamment de réduire le nombre de composants externes nécessaires au fonctionnement de la solution et d'en maîtriser davantage les mécanismes internes. Dès lors, la problématique centrale de ce projet s'énonce ainsi : ***comment concevoir et déployer une solution autonome de supervision et de sécurisation d'un serveur Ubuntu, permettant de suivre ses performances et de détecter les événements de sécurité tout en limitant la dépendance à des outils tiers ?***

Pour répondre à cette problématique, la création de la solution Sentinelle s'est faite de manière progressive et itérative. Le projet a évolué d'un ensemble initial de scripts Bash lancés à la main vers une solution complète, automatisée et modulaire. La version finale intègre la collecte en arrière-plan des métriques système (CPU, RAM, disque, réseau) via des services Systemd, le stockage persistant des données sous MySQL, la détection et le blocage des adresses IP malveillantes selon un mécanisme inspiré du fonctionnement de Fail2Ban, ainsi qu'un dashboard web associé à un espace d'administration sécurisé par un jeton d'authentification JWT.

Après une phase initiale de développement et de premiers tests sur un environnement Kubuntu 26.04, la solution a été déployée de manière automatisée sur une machine virtuelle Ubuntu Server pour sa validation en pré-production. L'ensemble du projet a ensuite été versionné et hébergé sur GitHub, puis conteneurisé sous Docker et Docker Compose afin de garantir sa portabilité avant son déploiement final dans le cloud.

Ce rapport est structuré selon un plan en trois chapitres. Le premier chapitre présente l'entreprise DEVOX, ses activités principales, son organisation ainsi que ses domaines d'intervention auprès de ses clients. Le deuxième chapitre est consacré au sujet du stage et détaille la conception de la solution Sentinelle de la version 1 à la version 4, ses protocoles de tests en pré-production et ses modalités de déploiement sur machine virtuelle et via Docker. Enfin, le troisième chapitre analyse les outils et supports techniques mis en œuvre, notamment la gestion des privilèges POSIX sous Ubuntu Server, l'orchestration Docker, ainsi qu'une étude comparative des solutions de référence ayant inspiré le projet (Fail2Ban, Icinga et Cacti).

D) L'entreprise DEVOX :

DEVOX est une société marocaine spécialisée dans les Technologies de l'Information, le développement web et mobile, créée en 2014. Basée à Rabat au Maroc, DEVOX se constitue d'une équipe d'une quinzaine de membres passionnés et chevronnés, diplômés des grandes écoles d'ingénieurs. Elle possède un savoir-faire et une expertise innovante et reconnue dans la conception, le développement, la mise en œuvre et la gestion de solutions IT avancées, particulièrement axées sur les applications B2C/Business To Consumer facilitant la mise en relation directe entre une entreprise et un consommateur final, la livraison, la prise de rendez-vous et tous les services annexes à ce type de plateforme. Enfin, l'entreprise est engagée à accompagner ses clients dans leur transformation numérique tout en assurant une performance durable et optimisée pour la digitalisation de n'importe quel processus métiers.

Les principales activités principales de l'entreprise DEVOX sont :

1. Développement de Logiciel sur mesure et Application Web et Mobiles
2. Gestion des Solution et des Infrastructures Cloud (DOCKER, AWS)
3. Audit de Sécurité IT et Analyse de Vulnérabilités et Protection des Données
4. Mise en œuvre des Stratégies DevOps
5. Conseil et Stratégie IT, Accompagnement de la Transformation Digitale (AMOA/AMOE) et Optimisation des processus métiers

Qui sont les principaux clients de l'entreprise DEVOX ?

Les clients de Devox se répartissent en 2 catégories de secteurs : secteur public et secteur privé.

Parmis les clients du secteur publics nous retrouvons :

- La Gendarmerie Royale du Maroc
- Le Ministère des Finances du Maroc
- L'Université Mohamed V de Rabat
- L'Université Abdelmalek Essaâdi de Tétouan
- l'ONOUSC (l'Office National des Œuvres Universitaires Sociales et Culturelles)
- l'ANEP (l'Agence Nationale des Équipements Publics)
- STAD (Skhirat-Témara Aménagement et Développement)

Parmis les clients du secteur privé nous retrouvons :

- Redal
- Le Cabinet Fikrat d'Expertise
- Orsys
- Injaze
- Kitea Geant

II) Sujet du stage et présentation du projet :

Le projet de ce projet de stage se divise en 2 parties : création de l'application et déploiement

Ce stage s'articule sur la mise en place d'une démarche DevSecOps pour la sécurisation du cycle de vie des applications web intégrant l'API Claude, le tout déployé sur le cloud.

Concernant le thème de l'application à déployer, il m'a été donné une certaine liberté de choix parmi 2 options :

- Déployer sur un serveur Ubuntu une application déjà existante développée par les ingénieurs de DEVOX
- Créer une application de zéro et la déployer par la suite sur un serveur Ubuntu

Afin de pratiquer le plus possible et améliorer mes compétences, mon choix s'est naturellement porté vers cette deuxième option. J'ai alors opté pour une application solution de monitoring pour serveur Ubuntu : **la solution Sentinelle**.

1 - Principe général :

L'idée générale de cet outil est la sécurisation et le monitoring des événements d'un serveur Ubuntu sans utiliser d'outils tiers avec une solution multi scripts en Bash. En effet, en cas de faille exploitée par des cybercriminels sur ces outils pourrait entraîner indirectement une vulnérabilité du serveur en exposant ce dernier via ces outils. C'est là que cette solution se démarque de celles préconçues et proposées par des outils tiers, car en effet cette dernière ne tourne qu'en local sur le serveur, n'est donc pas exposée facilement à des cybercriminels et les scripts qui la compose peuvent être adapté aux besoin du serveur cible sur lequel cette solution sera déployée.

2 - Conception, versions et tests en pré-production :

A - Conception et versions :

Au fur et à mesure de la conception il a fallu passer par différentes étapes afin d'obtenir une application de monitoring complète.

Sentinelle version 1 : la solution manuelle

Cette première version se compose d'une dizaine de scripts qui se lancent à la main selon un certain ordre :

- lancement du script d'installation pour s'assurer que tout est à jour
- lancement du script de scan
- lancement du script adapté à la sortie retourné par le script de scan

Le principal avantage de cette solution est que l'administrateur peut superviser le scan et lui-même gérer les informations. Mais cette solution est chronophage et peut laisser passer des informations essentielles. Il est donc nécessaire de faire en sorte que la solution puisse tourner d'elle-même en arrière-plan sans intervention.

Sentinelle version 2 : la solution en arrière plan

Cette version s'inscrit dans la continuité de la version 1. Cette fois-ci il suffit de lancer le script d'installation et toute la solution s'installe, se déploie et tourne en arrière plan sur le principe du **CI/CD**. Cela permet de ne pas avoir à lancer les scripts manuellement et de pouvoir inscrire toutes les informations remontées dans une base de données gérée par le **SGBD MySQL**.

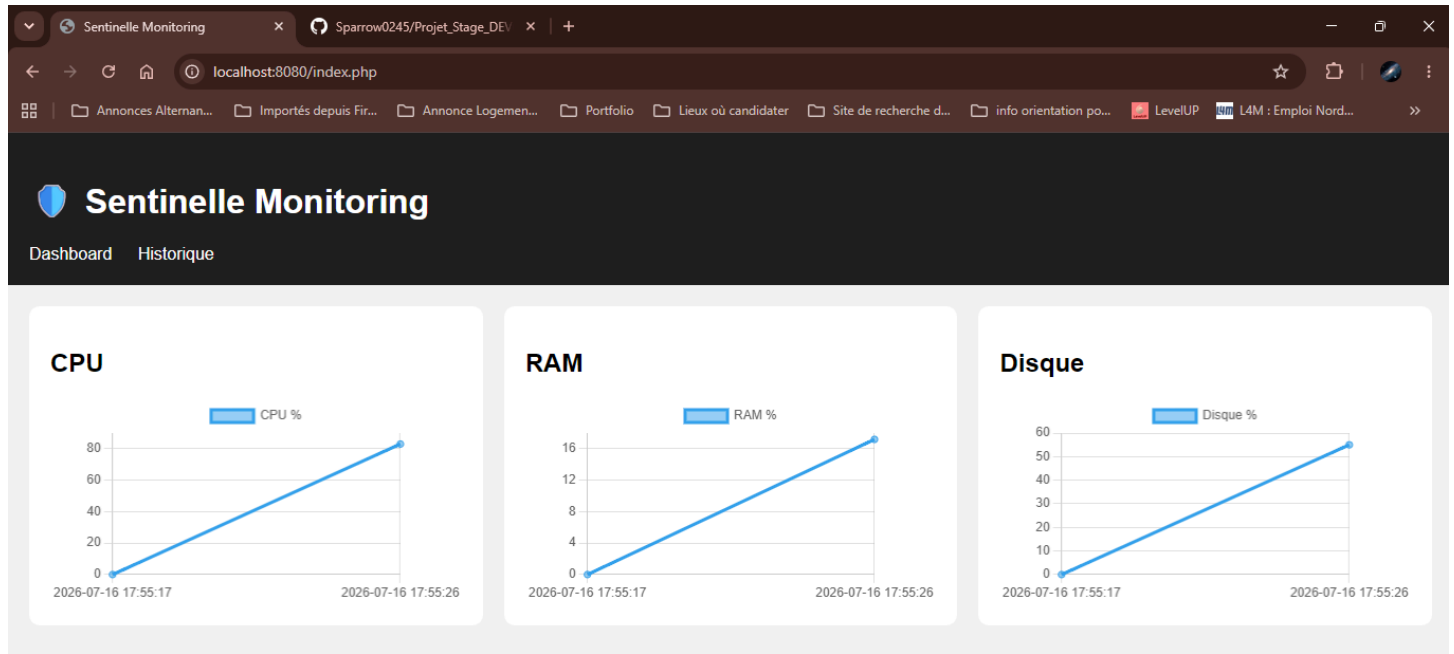
Afin de davantage prendre en compte l'aspect cybersécurité, certains ajouts ont été faits pour gérer les attaques en bloquant les adresses IP agressives par un système de prison (ou jail). En effet, si une adresse IP agit de manière suspecte ou tente des connexions répétées, via **SSH** par exemple, ou envoie un nombre considérable de requête au serveur pour le faire planter comme dans le cas d'une **attaque DDoS**, des scripts vont isoler cette adresse IP agressive et la placer sur une liste noire, l'empêchant ainsi de tenter toute connexion à l'avenir. C'est le modèle Fail2Ban.

Cependant, cette version peut être améliorée sur un point : la visualisation des informations. En effet, il est préférable d'avoir la possibilité de traiter visuellement les informations pour une bonne supervision.

Sentinelle version 3 : le dashboard simple

Afin de simplifier et d'améliorer l'utilisation de cet outil, la création d'un dashboard constitue une amélioration logique. Pour cela, il est nécessaire d'ajouter quelques scripts qui remonteront des informations supplémentaires en temps réel sur l'état du serveur comme la consommation de RAM, la consommation de disque, et la consommation de CPU. Le tout s'affiche sous forme de courbes dans une page web à l'adresse "<http://localhost:8080/>" (le port 8080 étant celui sur lequel écoute le serveur hébergé sur la VM Ubuntu Server pour ce projet). Pour cela, il est nécessaire de créer une interface web afin de visualiser ces informations en localhost.

Cette première version du dashboard permet d'avoir une vision globale sur les événements se déroulant sur le serveur en direct. Afin de compléter cette avancée, il peut être utile d'ajouter certains éléments de sécurité comme la remontée des alertes. Cela nous permet de passer sur une version plus complète de l'application de monitoring en y ajoutant des éléments de cybersécurité.



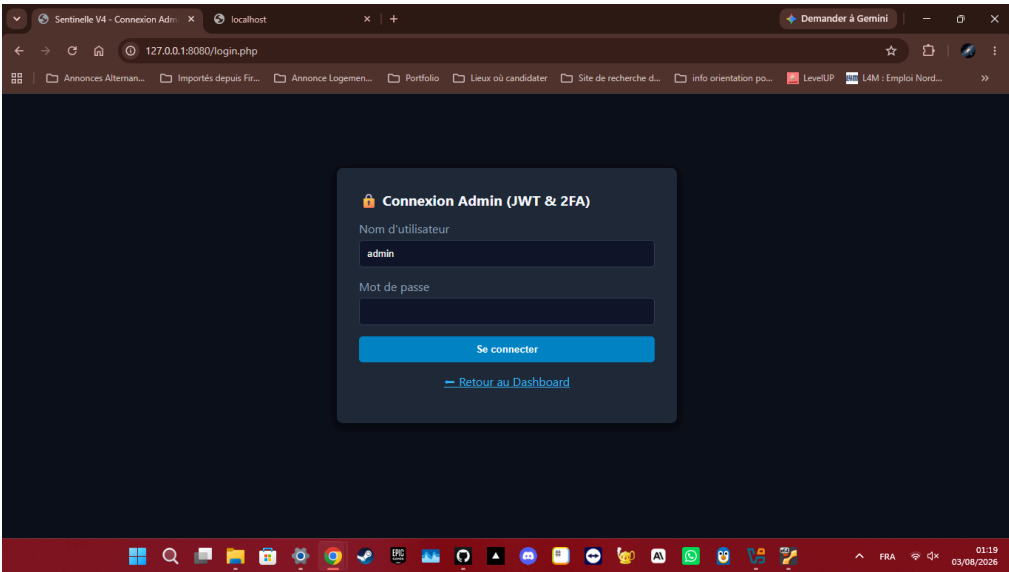
Dashboard de la Version 3.

Sentinelle version 4 : le dashboard complet

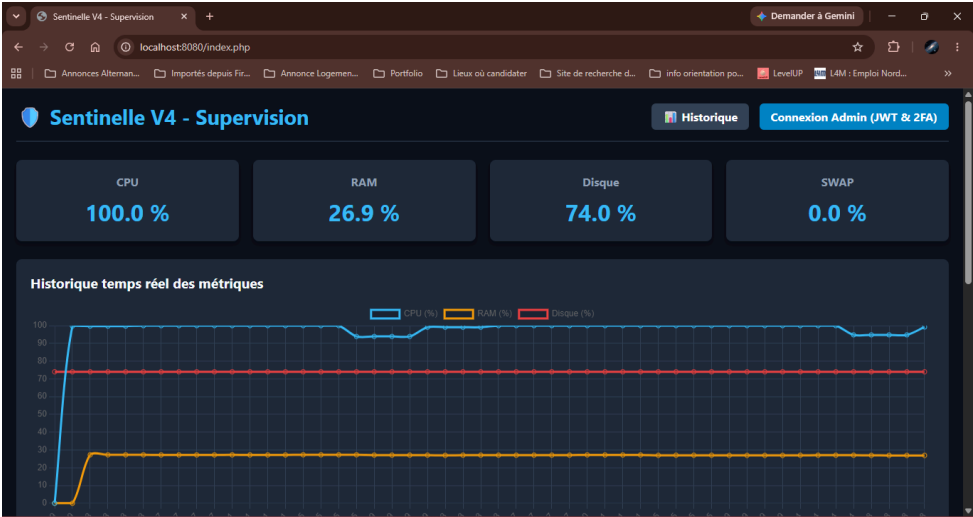
Pour cette version 4, nous entrons dans la phase finale du projet en se rapprochant d'une vraie application de monitoring. En effet, En plus de remonter les informations sur la consommation de disque, de RAM, et de CPU, cette fois-ci nous introduisons une vraie part de cybersécurité. En effet, désormais le dashboard affiche toutes les notifications de sécurité. Une nouveauté s'ajoute avec cette version : le dashboard administrateur. Cette page permet de visualiser, en plus des informations publiques :

- une page avec un profil administrateur plus complet avec authentification par **Token JWT**
- le journal détaillé des dernières événements et alertes
- les 50 derniers enregistrements de métriques.
- l'espace SWAP
- une page d'historique avec toutes les informations récoltées

Cette version plus complexe possède plusieurs pages de visualisation permettant ainsi de s'authentifier en profil administrateur, ou de traiter un type d'information en particulier.



Page de connexion à l'interface administrateur



Dashboard de la V4 de Sentinel

ID	Horodatage	Type	Sévérité	Message Détaillé
524	2026-08-18 12:09:58	CRITICAL	INFO	Le service système ssh est inactif ou en échec !
525	2026-08-18 12:09:58	CRITICAL	INFO	Le service ssh est inactif !
523	2026-08-18 12:09:57	CRITICAL	INFO	Le service mariadb est inactif !
521	2026-08-18 12:09:56	CRITICAL	INFO	Le service apache2 est inactif !
522	2026-08-18 12:09:56	CRITICAL	INFO	Le service système ssh est inactif ou en échec !
520	2026-08-18 12:09:55	CRITICAL	INFO	Le service système ssh est inactif ou en échec !

Dashboard du point de vu administratif

Cette version 4 marque une étape importante dans l'évolution du projet. Les différentes fonctionnalités ajoutées permettent de passer d'un simple outil de supervision à une application de monitoring plus complète, intégrant à la fois le suivi des performances du serveur et la visualisation des événements liés à la sécurité. La mise en place d'un espace administrateur sécurisé et de pages dédiées à l'historique des données apporte également une meilleure exploitation des informations collectées et facilite l'analyse de l'état du système sur le long terme.

B - Tests en pré-production :

Avant même de déployer cet outil sur un VM de test, il est utile de tester ces scripts en local sur une machine Linux (ici sous Kubuntu 26.04) afin de prendre en main les scripts.

Pour cette phase de pré-production localisée, l'objectif principal est de valider le fonctionnement unitaire et l'enchaînement des différents scripts Bash de l'application **Sentinelle**. Après l'attribution des droits d'exécution via la commande `chmod +x`, les scripts ont été exécutés et éprouvés sur le système hôte. L'automatisation des tâches de collecte est prise en charge par l'écosystème Systemd à l'aide du service `sentinelle-collector.service` et du timer `sentinelle-collector.timer`, garantissant une exécution régulière et autonome des traitements.

Cette architecture repose sur une séparation claire des responsabilités entre chaque composant :

- **Collecte ciblée des ressources** : les scripts dédiés (`collect_cpu.sh`, `collect_ram.sh`, `collect_disk.sh`, `collect_network.sh`, `collect_service.sh` et `collect_ssh.sh`), regroupés sous `collect_systeme.sh`, permettent de remonter les informations sur l'état du système.
- **Persistance et gestion des données** : le script `enregistrer_metrics.sh` formate les données collectées, puis le script `insertion_bdd.sh` se charge d'alimenter directement la base de données SQL.
- **Maintenance de l'historique** : le script `nettoyage_historique.sh` assure la purge périodique des anciennes données afin d'optimiser l'espace disque et de conserver des performances BDD fluides.
- **Exposition et affichage** : les API Web (`metrics.php` et `events.php`) interrogent la base de données pour alimenter les éléments visuels du dashboard (`index.php` et `historique.php`).

Cette étape de validation locale sur Kubuntu 26.04 s'est déroulée sans difficulté et a permis de vérifier le bon déroulement du cycle complet, de la capture de la métrique jusqu'à son affichage web. Bien que la partie gestion et déclenchement des alertes n'ait pas encore été sollicitée lors de ces premiers essais, la validation de la chaîne globale (collecte Bash, automatisation Systemd et stockage BDD) confirme la solidité de la base logicielle avant son déploiement dans un environnement virtuel dédié.

3 - Déploiement sur une VM de test Ubuntu Serveur :

Une fois l'architecture de la version 4 stabilisée, l'étape suivante consiste à valider le déploiement sur une machine réelle sous Ubuntu Server. L'objectif est de se rapprocher au maximum d'un environnement de production. Pour cela, un script maître (**lancement.sh**) orchestre l'exécution séquentielle de huit scripts Bash. Chacun des scripts est responsable d'une étape précise de l'installation.

Le premier met à jour le système et installe les dépendances : Java 21, PHP, MySQL, ainsi que les outils système nécessaires. Le second configure la base de données MySQL, avec création de l'utilisateur et des tables dédiées. Les scripts suivants déploient le backend Spring Boot vers **/opt/sentinel**, puis le frontend et sa configuration de connexion à la base.

Vient ensuite la configuration d'Apache, avec un reverse proxy vers l'API backend. Les services systemd sont alors activés, pour assurer la persistance du collector et du backend. Les permissions de sécurité sont appliquées : droits d'exécution sur les scripts, propriété **www-data** sur la racine web, restriction du fichier d'identifiants **MySQL**. Enfin, une phase de tests de recette vérifie la connexion à la base, la présence du collector, et l'insertion effective des métriques.

Ce script doit être exécuté avec les droits root. Il s'appuie exclusivement sur les composants déjà présents dans le dépôt. Cela permet de reproduire un déploiement complet et identique, sur n'importe quelle VM de test Ubuntu Server.

4 - Sauvegarde et hébergement sur Github :

L'ensemble du projet est versionné et hébergé sur un dépôt GitHub public. Cela assure une sauvegarde centralisée du code. C'est aussi un point d'accès unique pour le déploiement sur un nouveau serveur.

La procédure de mise en production est simple. Elle repose sur un clonage du dépôt, suivi de l'exécution du script de lancement. On récupère d'abord le code via **git clone**. On se positionne ensuite dans le répertoire du projet. On attribue les droits d'exécution au script principal. On le lance enfin, avec les privilèges administrateur.

Cette approche garantit un point important : le serveur de test, comme un futur serveur de production, peut être reconstruit à l'identique à partir du dépôt. Aucune sauvegarde locale ni transfert manuel de fichiers n'est nécessaire.

Le dépôt sert également de support de documentation. Il regroupe les différentes versions successives du projet, de la version 1 (un ensemble de scripts basique) jusqu'à la version 4 (la plateforme de supervision sécurisée), ainsi que les procédures et fichiers de configuration associés.

5 - Déploiement en Cloud avec Docker :

En parallèle du déploiement natif sur VM, une couche de conteneurisation Docker a été développée. Elle propose une méthode de mise en production alternative, isolée, et indépendante d'un fournisseur cloud particulier.

Cette architecture repose sur quatre conteneurs distincts. Le premier, *sentinelle-collector*, exécute les scripts Bash de collecte. Il s'appuie sur le namespace PID et le réseau de l'hôte, ainsi que sur des montages en lecture seule de `/`, `/proc`, `/sys`, `/run`, `/var/log` et `/etc`. Cela lui permet de superviser le serveur hôte depuis l'intérieur d'un conteneur. Le second, *sentinelle-backend*, fait tourner l'application Spring Boot sous Java 21. Il expose l'API REST sur le port interne 8080. Le troisième, *sentinelle-web*, repose sur Apache et PHP. Il sert l'interface et joue le rôle de reverse proxy vers le backend. Le dernier, *sentinelle-db*, est une base MySQL optionnelle. Elle n'est activée que si la variable `DB_MODE` est réglée sur `local` — une base externe déjà existante peut sinon être utilisée, via les variables `EXTERNAL_DB_*`.

Le déploiement se fait en quelques étapes. On copie d'abord le fichier *.env.example* en *.env*. On modifie ensuite les identifiants sensibles : mot de passe de la base, secret JWT. On exécute enfin le script *lancement-docker.sh*. Celui-ci vérifie la présence de Docker et de Docker Compose, construit les images, démarre les conteneurs, puis affiche leur état.

Des scripts complémentaires permettent ensuite de piloter la solution au quotidien : *statut-docker.sh*, *logs-docker.sh*, *redemarrage-docker.sh*, *arret-docker.sh*. La persistance des données locales repose sur un volume Docker dédié, *sentinelle-db-data*. Il n'est supprimé que par un *docker compose down -v* explicite. Cette approche permet à la solution Sentinelle de fonctionner sur n'importe quel serveur Linux disposant de Docker, sans dépendance à un environnement cloud spécifique.

III) Présentation des outils et supports :

Dans un souci de cybersécurité et de continuité des services, l'utilisation de services ou d'outils tiers pour ce projet est limitée au strict nécessaire.

A - Un serveur de production sous Ubuntu :

Les droits des utilisateurs :

Afin de structurer les activités sur le serveur, chacun des utilisateurs doit avoir accès uniquement aux droits qui lui sont réellement nécessaires.

Sur un système GNU/Linux comme Ubuntu Server, la gestion des accès repose sur une distinction stricte entre les tâches d'utilisation courante et les opérations d'administration système.

Pour éviter l'utilisation permanente d'un compte superutilisateur (*root*), potentiellement risquée pour l'intégrité de l'infrastructure, Ubuntu privilégie une délégation contrôlée des privilèges au moyen de la commande *sudo*. Ce mécanisme permet d'attribuer temporairement des droits d'administration à des utilisateurs autorisés pour des actions spécifiques (installation de paquets, modification de configurations réseau ou gestion des services) tout en traçant les exécutions.

Cette sécurité est complétée au niveau du système de fichiers par le modèle POSIX de gestion des propriétaires et des permissions. Chaque ressource (fichier ou répertoire) est associée à un utilisateur propriétaire (**u**), un groupe propriétaire (**g**) et aux autres utilisateurs du système (**o**). À chacune de ces trois catégories sont attribués trois types de droits fondamentaux :

- La lecture (**r**) : elle permet d'afficher le contenu d'un fichier ou de lister le contenu d'un répertoire.
- L'écriture (**w**) : elle autorise la modification d'un fichier ou la création, le renommage et la suppression d'éléments au sein d'un répertoire.
- L'exécution (**x**) : elle conditionne le lancement des scripts ou des exécutables, et permet d'accéder au contenu d'un répertoire (**cd**).

La configuration fine de ces droits s'effectue via des outils en ligne de commande tels que **chmod** (en notation symbolique ou octale) et **chown** pour le transfert de propriété. Dans le cadre du serveur de production hébergeant l'application Sentinelle, l'application rigoureuse de ces règles garantit le principe du moindre privilège. Les scripts de collecte, les services d'arrière-plan et le serveur web s'exécutent ainsi avec le niveau d'accès strictement nécessaire, évitant qu'une éventuelle compromission d'un composant applicatif n'accorde un accès critique à l'ensemble du système d'exploitation.

B - Docker :

Dans le cadre du déploiement de la solution de monitoring Sentinelle V4, une couche de conteneurisation basée sur Docker et Docker Compose a été mise en place afin d'isoler les composants, d'assurer la portabilité de l'application et de simplifier son déploiement global. Cette approche offre un déploiement automatisé tout aussi simple que le script de lancement initial de la solution, tout en garantissant la reproductibilité de l'environnement applicatif.

1. Architecture multi-conteneurs :

L'architecture conteneurisée repose sur une séparation claire des responsabilités entre plusieurs services spécialisés :

- **sentinelle-collector** : Développé à partir d'une image Ubuntu 24.04, ce conteneur exécute les scripts de collecte en Bash. Pour superviser le serveur Linux hôte, il utilise le réseau et le namespace PID de l'hôte, bénéficie de privilèges étendus et intègre des montages en lecture seule de la racine et des dossiers système (**/**, **/proc**, **/sys**, **/run**, **/var/log** et **/var/run/docker.sock**). Il intègre également des wrappers s'appuyant sur **nsenter** pour exécuter des commandes comme **systemctl** ou **journctl** directement dans le contexte du système hôte.
- **sentinelle-web** : Ce conteneur héberge l'interface web de supervision sous un serveur Apache avec PHP.
- **sentinelle-db** : S'appuyant sur une image MariaDB 11.4, ce service assure la gestion et le stockage persistant du schéma de base de données **sentinelle** en déploiement local.

2. Flexibilité de la base de données et persistance :

La solution propose deux modes de fonctionnement configurables via la variable **DB_MODE** dans le fichier d'environnement :

- **Mode local (DB_MODE=local)** : La base de données MariaDB est déployée automatiquement sous forme de conteneur. La persistance des données est garantie par le volume Docker nommé **sentinelle-db-data**, évitant toute perte de données lors de l'arrêt des conteneurs.
- **Mode externe (DB_MODE=external)** : Permet de connecter directement le collecteur et l'interface web à une base de données MySQL/MariaDB préexistante sur un autre serveur, annulant le démarrage du conteneur **sentinelle-db** local.

3. Orchestration, automatisation et sécurité :

L'exploitation du déploiement est entièrement automatisée grâce à une série de scripts Bash dédiés situés dans le sous-dossier **scripts/** :

- **lancement-docker.sh** : Vérifie les prérequis Docker et Docker Compose, contrôle la configuration du fichier **.env**, construit les images et lance les conteneurs selon le mode de base de données sélectionné.
- **statut-docker.sh**, **logs-docker.sh**, **redemarrage-docker.sh** et **arret-docker.sh** : Permettent de consulter l'état des services, d'analyser la consommation des ressources, de visionner les journaux d'exécution et de gérer le cycle de vie des conteneurs.

Sur le plan de la sécurité, la gestion des secrets (mots de passe de base de données, paramètres d'accès) est centralisée dans un fichier d'environnement **.env** généré à partir de **.env.example** et exclu du suivi de version à l'aide du fichier **.dockerignore**.

C - Des scripts pour la supervision et le monitoring :

Afin de veiller au bon fonctionnement du système sans utiliser d'outils tiers, la mise en place d'une solution de monitoring à l'aide de scripts en Bash tournant exclusivement en local sur le serveur Linux constitue une alternative efficace. Le but est de mettre en place une solution toute aussi efficace qu'avec des outils mais ici reproduite avec des scripts bash propre au serveur. Pour cela, il est possible de s'inspirer du modèle de solutions préexistantes.

1 - Le modèle Fail2Ban :

Le principe du modèle Fail2Ban repose sur les principes suivant :

- **l'analyse des logs de divers services (SSH, Apache, etc...)** : il s'agit d'une lecture continue et en temps réel des fichiers journaux système et applicatifs. Chaque requête, tentative d'accès ou événement généré par un service réseau est consigné puis scruté afin d'isoler l'activité normale des anomalies.

- **la recherche de correspondance avec des schémas d'attaques configurés à l'avance (tentatives de connexion répétées, envoi massif de requêtes sur le serveur, etc...) :** le système applique des expressions régulières définies dans la configuration pour identifier des motifs suspects. Il comptabilise l'occurrence de ces événements sur une fenêtre temporelle donnée afin d'évaluer le niveau de menace.
- **analyse des logs pour déceler les tentatives répétées de connexion infructueuse (fichiers journaux) :** l'outil surveille spécifiquement les échecs d'authentification récurrents provenant d'une même source. Si le nombre d'échecs dépasse le seuil toléré dans le temps imparti, l'événement est immédiatement détecté comme une forme d'attaque.
- **exécution d'actions en réponse en cas de correspondance avec un schéma d'attaque (bannissement des adresses IP agressives avec ajouts de règle au pare-feu) :** dès qu'un comportement malveillant ou suspect est confirmé, le démon déclenche l'exécution de scripts de remédiation adapté à la menace détectée. Ces scripts modifient les tables du pare-feu (comme iptables ou nftables) pour bloquer le trafic entrant de l'adresse IP incriminée pendant une durée paramétrable.

Tout cela se résume à **3 principaux concepts** :

- **les prisons (jails) :** elles constituent le noyau de la configuration en associant un filtre donné à une ou plusieurs actions. Une prison définit également les seuils de tolérance, notamment le nombre maximal de tentatives échouées autorisées (*maxretry*), la fenêtre d'observation (*findtime*) et la durée d'interdiction d'accès (*bantime*).
- **les filtres (filters) :** ils regroupent les expressions régulières (*failregex*) qui permettent d'analyser la structure textuelle des fichiers de logs. Ils déterminent avec précision quelles lignes de texte correspondent à un échec d'authentification ou à une tentative d'intrusion.
- **les actions :** elles décrivent les commandes système exécutées lors du déclenchement ou de la levée d'une alerte. Cela comprend l'ajout et le retrait de règles dans le pare-feu, l'envoi d'e-mails d'avertissement à l'administrateur ou la création d'enregistrements spécifiques dans les journaux système.

2 - Le principe Icinga :

Le principe du modèle Icinga utilisé dans ce projet repose sur les principes suivant :

- **le contrôle régulier et automatisé de l'état des équipements et des services :** le système effectue des vérifications planifiées à intervalles réguliers sur l'ensemble des hôtes et des applications pour s'assurer de leur disponibilité et de leur bon fonctionnement.
- **l'exécution de sondes indépendantes pour évaluer des métriques précises :** le noyau fait appel à de petits programmes ou scripts autonomes (*plugins*) dédiés chacun à la mesure d'un indicateur spécifique, comme la charge processeur, l'occupation de la mémoire RAM, le taux d'utilisation des disques ou le temps de réponse HTTP.

- **l'évaluation des résultats selon des seuils prédéfinis pour attribuer un état standardisé** : chaque mesure retournée par une sonde est comparée à des valeurs limites d'avertissement (*warning*) et de criticité (*critical*), produisant un statut clair parmi quatre états possibles (**OK**, **WARNING**, **CRITICAL**, **UNKNOWN**).
- **la gestion des alertes** : dès qu'une transition d'état anormal est détectée, le système enregistre l'événement dans la base de données et cette information est consultable sur le dashboard.

Tout cela se résume à 3 principaux concepts :

- **les hôtes (hosts)** : ils représentent les entités physiques ou virtuelles supervisées (serveurs, routeurs, commutateurs). Chaque hôte est défini par son adresse réseau (IP) et regroupe l'ensemble des contrôles qui lui sont rattachés.
- **les services (services)** : ils désignent les composants logiciels, les métriques système ou les fonctionnalités réseau surveillés sur un hôte donné. Chaque service est associé à une sonde spécifique chargée d'en évaluer régulièrement la santé.
- **les sondes et les états (checks & states)** : les sondes sont des scripts d'exécution qui renvoient un code de sortie numérique (**0** pour **OK**, **1** pour **WARNING**, **2** pour **CRITICAL**, **3** pour **UNKNOWN**) accompagné d'un texte explicatif. Ces codes permettent d'unifier la gestion de la supervision sur l'ensemble de l'infrastructure.

3 - Le modèle Cacti :

Le principe du modèle Cacti repose sur les principes suivant :

- **la collecte périodique de métriques de performance système et réseau** : l'outil extrait régulièrement des valeurs chiffrées sur les équipements surveillés, telles que le débit des interfaces réseau, le nombre de processus actifs ou la consommation d'espace disque.
- **l'interrogation distante ou locale à l'aide de protocoles standards et de scripts** : la récupération des données s'effectue principalement via le protocole SNMP pour les équipements réseau, ou par le biais de scripts exécutables locaux (en Bash dans le cas présent) pour obtenir des indicateurs spécifiques.
- **l'archivage optimisé des données pour garantir un suivi historique à long terme** : les données collectées sont stockées dans des structures de fichiers spéciales qui agrègent progressivement les mesures anciennes afin de conserver un historique étendu sans augmenter l'espace disque consommé.
- **la restitution graphique des indicateurs sous forme de courbes temporelles** : la solution convertit les séries temporelles accumulées en graphiques visuels clairs permettant d'analyser l'évolution des performances, d'identifier les pics de charge et d'anticiper les besoins en capacité.

Tout cela se résume à 3 principaux concepts :

- **le poller (collecteur)** : c'est l'agent d'ordonnancement qui s'exécute à intervalles réguliers (généralement toutes les 5 minutes) pour lancer l'ensemble des requêtes de collecte et transmettre les résultats au moteur de stockage.
- **la base RRDTool (Round Robin Database)** : c'est le système de stockage circulaire sous-jacent. Il garantit une empreinte disque fixe et constante pour chaque métrique, en consolidant au fil du temps les données précises en moyennes temporelles (horaires, quotidiennes, mensuelles).
- **les graphiques et modèles (graphs & templates)** : ils constituent la couche de présentation et d'organisation. Les modèles définissent la structure des données à collecter, tandis que les moteurs de rendu traduisent ces données en représentations visuelles personnalisables pour l'administrateur.

En conclusion, l'étude de ces trois modèles permet de poser les bases d'une architecture de supervision complète, autonome et entièrement réalisée en Bash. Le modèle Fail2Ban apporte la **dimension réactive** grâce à l'**analyse des journaux** système et au déclenchement d'actions automatisées face aux anomalies. La philosophie d'Icinga offre une structure rigoureuse pour **contrôler régulièrement l'état des services** à l'aide de sondes et d'un système d'états standardisés. Enfin, l'approche de Cacti garantit la **collecte régulière de métriques chiffrées pour le suivi des performances dans le temps**. L'association de ces principes permet de concevoir une solution locale sur mesure, à la fois performante, peu gourmande en ressources et parfaitement adaptée aux besoins du serveur.

D - MySQL pour la base de données :

Afin de pouvoir stocker toutes les informations remontées par les scripts et pour pouvoir les afficher sur le dashboard, il est nécessaire de les stocker dans une base de données. La machine de production étant sous Ubuntu Server, plusieurs 2 choix s'offraient à nous parmi les SGBD : MySQL ou MongoDB.

1. Étude comparative et justification du choix du SGBD

Dans le cadre de la conception du système de supervision Sentinelle, un choix stratégique devait être fait concernant le système de gestion de base de données.

D'un côté, MongoDB repose sur un modèle NoSQL orienté document. Il apporte une grande souplesse pour stocker des données non structurées sous forme de documents BSON/JSON sans imposer de schéma rigide. Cependant, cette flexibilité implique de devoir gérer la cohérence et l'intégrité des liaisons de données au niveau applicatif.

D'un autre côté, MySQL est un SGBD relationnel classique et éprouvé, garantissant le respect strict des propriétés ACID (Atomicité, Cohérence, Isolation, Durabilité). Il s'appuie sur une organisation rigoureuse en tables et gère nativement l'intégrité référentielle par le biais de clés étrangères.

Le choix s'est naturellement porté sur MySQL. En effet, l'ensemble des données collectées par l'application (hôtes, métriques système, état des services, journaux d'événements et identifiants utilisateurs) présente une structure clairement définie et intrinsèquement relationnelle. L'utilisation de MySQL permet de verrouiller la cohérence des données au niveau du SGBD tout en s'appuyant sur un outil simple d'utilisation, rapidement déployable et déjà maîtrisé dans l'environnement de travail.

2. Architecture relationnelle et description des tables

La base de données, nommée *sentinelle*, a été conçue selon une structure relationnelle normalisée afin de minimiser la répétition d'informations et d'optimiser les performances de consultation du tableau de bord.

L'organisation repose sur cinq tables principales :

- La table *hosts* constitue le cœur de la base. Elle recense chaque machine surveillée en enregistrant son identifiant unique *id*, son nom d'hôte *hostname*, son adresse IP *ip_address*, son statut général *status* via une énumération (*online*, *offline*, *warning*) et son horodatage de création *created_at*.
- La table *metrics* enregistre la consommation des ressources matérielles des hôtes. Elle contient une clé primaire *id* dimensionnée en *BIGINT* pour absorber une importante volumétrie de relevés sur le long terme, la clé étrangère *host_id*, ainsi que les indicateurs *cpu_usage*, *ram_usage*, *disk_usage* et *swap_usage* stockés en type *DECIMAL(5,2)* pour préserver la précision des pourcentages. Chaque enregistrement est horodaté par le champ *timestamp*.
- La table *services_status* suit l'état des services système supervisés. Outre sa clé primaire *id* et la clé étrangère *host_id*, elle enregistre le nom du service *service_name*, son état *status* (*active*, *inactive*, *failed*) et la date de dernière mise à jour *updated_at*.
- La table *events* sert de journal d'audit pour les alertes générées par le système. Elle comprend la clé primaire *id* en *BIGINT*, la clé étrangère *host_id*, le niveau de sévérité *type* (*INFO*, *WARNING*, *CRITICAL*), le message détaillé *message* et la date de création *created_at*.
- La table *users* gère les accès à l'interface web de supervision. Elle contient le nom d'utilisateur unique *username*, le mot de passe haché *password_hash*, la clé secrète pour le double facteur *totp_secret* et le rôle d'accès *role* (*admin* ou *viewer*).

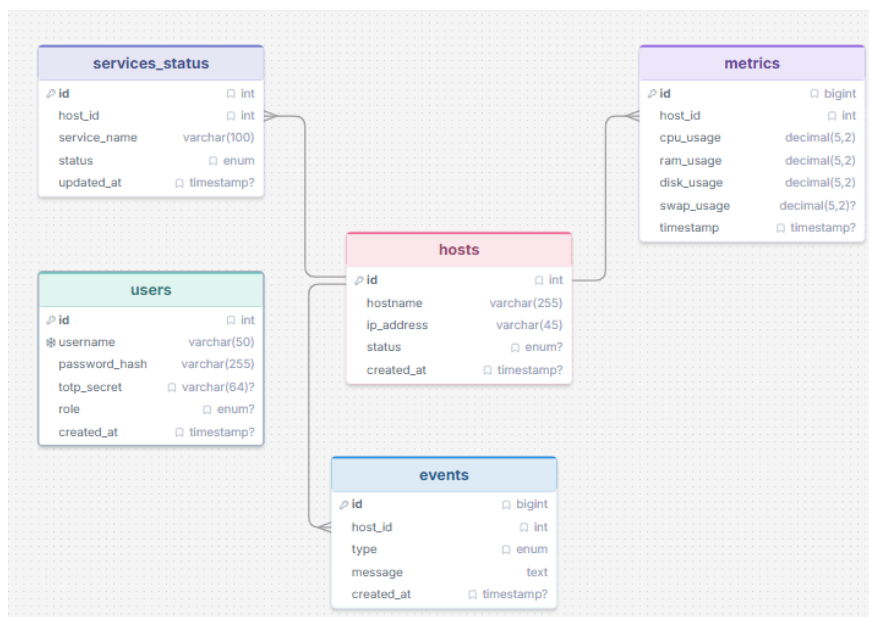


Schéma relationnel de la base de données Sentinelle.

Comme l'illustre le schéma relationnel, la table **hosts** se trouve au centre de la structure. Les tables **metrics**, **services_status** et **events** y sont reliées par une relation de 1 à N via la clé étrangère **host_id**. De son côté, la table **users** demeure indépendante, ce qui permet d'isoler la couche d'authentification des données applicatives de surveillance.

3. Optimisation technique et intégration DevSecOps

Dans une démarche DevSecOps favorisant la sécurité dès la conception (Security by Design), plusieurs choix techniques et configurations de durcissement ont été appliqués sur la base de données :

L'ensemble des tables utilise le moteur de stockage **InnoDB**. Ce choix garantit la prise en charge native des transactions ACID et assure la cohérence des données grâce au support des contraintes d'intégrité référentielle.

Une contrainte de suppression en cascade (**ON DELETE CASCADE**) est configurée sur toutes les clés étrangères **host_id**. En cas de retrait d'un hôte dans la table **hosts**, toutes les métriques, statuts de services et événements associés sont supprimés automatiquement. Cela évite la présence de données orphelines et facilite la maintenance automatisée de la base.

L'encodage retenu est le **utf8mb4** associé à la collation **utf8mb4_unicode_ci**. Il assure le support complet de l'ensemble des jeux de caractères tout en protégeant l'application contre les failles d'injection liées aux conversions d'encodage.

Concernant la sécurité des accès, aucun mot de passe n'est stocké en clair. Le champ **password_hash** enregistre uniquement des empreintes chiffrées générées via l'algorithme fort BCrypt. L'intégration du champ **totp_secret** permet de gérer la double authentification (2FA) basée sur des jetons temporels à usage unique. Le contrôle d'accès basé sur les rôles (RBAC) via le champ **role** limite les privilèges d'administration aux seuls profils autorisés. Enfin, le SGBD est configuré pour n'écouter que sur l'interface locale (**127.0.0.1**), interdisant ainsi toute exposition directe du service MySQL sur le réseau extérieur.

IV) Conclusion :

Ce stage de fin de BUT d'une durée de 13 semaines, effectué au sein de l'entreprise DEVOX à Rabat, constitue la dernière étape et l'aboutissement de ma troisième année de BUT Informatique, spécialité Déploiement d'Applications Communicantes et Sécurisées, réalisée à l'IUT de Villeneuve-d'Ascq. Pleinement ancrée dans la philosophie de ce parcours, cette immersion professionnelle avait pour objectif central d'inscrire mes réalisations dans une démarche DevSecOps rigoureuse, couvrant l'ensemble du cycle de vie d'une application, de sa conception à sa sécurisation et son déploiement conteneurisé.

1. Synthèse des réalisations techniques et valeur ajoutée pour DEVOX :

La mission principale qui m'a été confiée au cours de ce stage a porté sur le développement de **Sentinelle**, une plateforme autonome de monitoring et de sécurisation dédiée aux environnements serveur Ubuntu. Dès mon arrivée dans l'entreprise, j'ai pris l'initiative de proposer ce sujet d'application de monitoring, une idée qui a été particulièrement bien accueillie et appréciée par mon tuteur professionnel pour son utilité directe au sein de la structure.

Le projet s'est déroulé selon une méthodologie itérative rigoureuse :

- **Versions initiales (V1 à V2)** : Mise en place d'une première batterie de scripts Bash exécutés ponctuellement, permettant de valider la collecte brute des métriques système et de poser les bases de la logique de surveillance.
- **Version intermédiaire (V3)** : Structuration d'un module d'analyse des journaux d'accès via un dashboard afin de détecter et de bloquer automatiquement les adresses IP suspectes sur le modèle de l'outil Fail2Ban, couplée à une persistance des données dans une base de données MySQL.
- **Version finale (V4)** : Finalisation d'un tableau de bord web d'administration ergonomique, sécurisé par des jetons JWT et une authentification à deux facteurs (TOTP/2FA), tout en assurant une collecte continue en arrière-plan via des services Systemd.

Afin de garantir la portabilité et la pérennité de l'outil pour les équipes de DEVOX, l'application a été entièrement industrialisée. D'une part, un script d'installation automatique (***lancement.sh***) permet un déploiement natif rapide sur une machine nue, avec un suivi de version complet archivé sur un dépôt public GitHub. D'autre part, la solution a été découpée et conteneurisée sous Docker et Docker Compose en quatre microservices distincts (***sentinelle-collector***, ***sentinelle-web***, ***sentinelle-db*** et ***sentinelle-backend***), la rendant immédiatement prête pour une intégration dans des architectures cloud modernes.

L'apport de cette réalisation est concret pour DEVOX : l'entreprise bénéficie désormais d'un socle interne, souverain et évolutif pour surveiller et protéger ses serveurs sans dépendre d'outils propriétaires complexes ou onéreux.

2. Analyse critique, limites techniques et pistes d'évolution :

La conduite d'un projet de cette ampleur impose un regard critique sur les défis surmontés et sur les limites actuelles de la solution.

Le principal verrou technique a résidé dans le compromis entre l'isolation des conteneurs Docker et la nécessité d'accéder aux métriques réelles du serveur hôte. L'étanchéité native de la conteneurisation empêchait initialement le collecteur d'accéder aux données bas niveau de la machine hôte. Ce problème a été résolu en configurant des montages en lecture seule de dossiers système critiques (`/`, `/proc`, `/sys`), en exploitant le namespace PID ainsi que le réseau de l'hôte, et en adaptant les commandes via des utilitaires ciblés comme `nsenter`.

Néanmoins, une analyse objective du travail livré met en évidence deux limites principales :

1. **Périmètre d'action monoposte** : Dans sa version actuelle, la solution est dimensionnée pour superviser un serveur ou un nœud local unique. Elle ne permet pas encore une gestion centralisée et multi-nœuds au sein d'un cluster étendu.
2. **Système de notification réactif** : La consultation des alertes repose aujourd'hui principalement sur le tableau de bord web et la consultation des journaux système, sans intégration native d'alertes poussées en temps réel (ex. Webhook, notification par courriel ou messagerie instantanée).

Dans la perspective d'une suite à donner au projet après mon stage, j'ai proposé à l'équipe technique de DEVOX une feuille de route pour les évolutions futures :

- Développer une architecture maître-esclave permettant à une instance centrale Sentinelle de remonter les métriques de plusieurs serveurs distants.
- Intégration d'un module d'alerte immédiate (envoi de notifications par e-mail ou Webhook Slack/Discord lors d'un pic de charge ou d'un blocage d'IP).
- Étendre la compatibilité des scripts de collecte à d'autres distributions Linux (Debian, Red Hat Enterprise Linux).

3. Bilan personnel, compétences et intégration :

Sur le plan professionnel et individuel, cette expérience chez DEVOX m'a permis de franchir un cap significatif dans la maîtrise d'outils et de méthodologies complexes. J'ai pu renforcer de façon pragmatique mes compétences en administration système Linux, en orchestration de conteneurs avec Docker, ainsi qu'en sécurité applicative et réseau (gestion du contrôle d'accès, hachage BCrypt, authentification forte).

Ce stage m'a également permis de développer une réelle autonomie dans la conduite d'un projet logiciel, de la phase de définition des besoins jusqu'à la livraison d'un produit opérationnel.

Sur le plan humain, l'accueil de l'équipe et l'accompagnement de mon tuteur ainsi que du patron de l'entreprise au sein de l'entreprise DEVOX à Rabat au Maroc m'ont permis de comprendre et de découvrir un mode de travail différent de celui que je connaissais de part mes précédentes expériences. Cette nouvelle dynamique pourra être un plus non négligeable dans ma future vie professionnelle.

4. Perspective comparative : du BTS CIEL au BUT Informatique

Ayant intégré la formation directement en troisième année de BUT après l'obtention d'un BTS CIEL (Cybersécurité, Informatique et réseaux, Électronique), il est particulièrement enrichissant de mesurer le chemin parcouru en comparant cette expérience chez DEVOX à mon stage de première année de BTS effectué au sein de l'entreprise Nautech Shipyard à La Ciotat.

Lors de mon stage de BTS CIEL, mes tâches relevaient principalement de la configuration, de la sécurité opérationnelle et de l'administration réseau appliquée :

- Configuration d'équipements de sécurité réseau (pare-feu Stormshield).
- Déploiement d'agents de collecte Elastic Search rattachés à une infrastructure de supervision Centreon déjà en place.

Cette première immersion en BTS CIEL s'inscrivait dans un cadre technique prédéfini où les choix d'architecture étaient déjà arrêtés. À l'inverse, ce stage de troisième année de BUT Informatique, spécialité Déploiement d'Applications Communicantes et Sécurisées, m'a demandé d'adopter une posture d'ingénierie globale et de prendre la responsabilité complète de la conception, du développement, du durcissement et du déploiement d'une solution sur mesure.

La complémentarité entre ces deux cursus est évidente : les compétences solides acquises en systèmes, réseaux et sécurité lors du BTS CIEL ont constitué le socle indispensable pour appréhender avec aisance les problématiques complexes d'architecture logicielle, de conteneurisation et de démarche DevSecOps abordées en BUT Informatique, spécialité Déploiement d'Applications Communicantes et Sécurisées.

Cette expérience confirme mon attrait pour les enjeux de la cybersécurité et a été très enrichissante professionnellement grâce à tout ce que j'ai appris pendant ce stage.

Cette nouvelle immersion professionnelle valide la continuité de mon parcours et confirme mon orientation vers les métiers de la cybersécurité.

VI) Lexique :

DevSecOps : approche du développement de logiciel qui intègre la sécurité à chaque phase du cycle de vie DevOps.

DevOps : approche permettant la coordination et la collaboration des rôles autrefois cloisonnés (développement, opérations informatiques, ingénierie qualité et sécurité) pour créer des produits plus performants et plus fiables.

SSH : le protocole SSH (Secure Shell) permet l'établissement d'une connexion chiffrée à distance entre des ordinateurs. Il permet également l'utilisation de la tunnellation.

Logs : Les fichiers log sont des fichiers de données (généralement sous forme de texte) générés par des appareils, des réseaux, des applications et des systèmes d'exploitation, contenant des informations enregistrées sur leurs activités, opérations et schémas d'utilisation. Ils constituent l'historique principal de tout ce qui se passe dans un environnement et fournissent des données cruciales sur les événements, les processus, la sécurité, les indicateurs de performance et l'activité des utilisateurs.

CI/CD : Le CI/CD relève du processus DevOps (la réunion des équipes de développement et des opérations) et allie les pratiques d'intégration continue et de livraison continue. Le CI/CD automatise une grande partie voire la totalité de l'intervention humaine manuelle traditionnellement nécessaire pour faire passer un nouveau code d'une validation à la production, ce qui englobe les phases de compilation, de test (y compris les tests d'intégration, les tests unitaires et les tests de régression) et de déploiement, ainsi que le provisionnement de l'infrastructure. Avec un pipeline CI/CD, les équipes de développement peuvent apporter des modifications au code, qui sont ensuite automatiquement testées et transférées en vue de la livraison et du déploiement.

Authentification par Token JWT (ou authentification par jeton) : L'authentification par jeton est un moyen de confirmer l'identité d'un utilisateur ou d'un appareil. Elle repose sur la vérification de la possession par l'entité en question d'un jeton préalablement émis.

API : Une API est un ensemble de définitions et de protocoles qui facilite la création et l'intégration de logiciels d'applications

API REST : Une API REST est une [interface de programmation d'application \(API\)](#) qui respecte les principes de conception du style d'architecture REST, abréviation de « REpresentational State Transfer ». Ce style regroupe un ensemble de règles et de consignes pour la création d'API web.

Plugin : Un plugin est un composant logiciel qui ajoute une fonctionnalité spécifique à un programme informatique ou à un navigateur web existant. Les plugins sont conçus pour étendre les capacités de l'application hôte sans qu'il soit nécessaire de modifier son code principal. Ils peuvent être utilisés à diverses fins, telles que l'ajout de nouvelles fonctionnalités, l'amélioration des performances, le renforcement de la sécurité ou l'intégration avec des services tiers.

SGBD : Un système de gestion de base de données (SGBD) est un logiciel système permettant aux utilisateurs et programmeurs de créer et de gérer des bases de données

SQL : SQL (pour Structured Query Language, prononcé « S-Q-L ») est un langage conçu pour interagir avec les bases de données relationnelles. Il permet d'effectuer des tâches telles que la mise à jour et la récupération de données à partir de bases de données.

Base de données SQL : Une base de données SQL, également connue sous le nom de base de données relationnelle, est un ensemble structuré de données organisées en tables, qui peuvent être traitées à l'aide d'un langage de requête structuré

NoSQL : Le terme « NoSQL » désigne les différents types de bases de données non relationnelles. Ces bases de données stockent les données dans un format différent. Toutefois, les bases de données NoSQL peuvent être interrogées à l'aide d'API en langage idiomatique, de langages déclaratifs et de langages de requête par exemple, ce qui explique pourquoi elles sont également considérées comme des bases de données « pas seulement SQL ».

Base de données NoSQL : Face à l'essor massif de la quantité d'informations dans le monde numérique, les bases de données (BDD) non relationnelles (ou NoSQL pour Not only SQL), se présentent comme une solution alternative aux systèmes relationnels classiques. Offrant une grande flexibilité, ces dernières sont aussi bien adaptées à la gestion des services d'analytique et au big data. Elles se distinguent par leur **capacité à traiter efficacement des données non structurées**, qui constituent un défi majeur pour les bases de données traditionnelles.

Clé primaire (Primary Key) : On appelle « clé primaire » une ou plusieurs colonnes d'une table de base de données dont les valeurs identifient de manière unique chaque ligne ou enregistrement.

Clé étrangère (Foreign Key) : Une clé étrangère est un concept critique dans les bases de données relationnelles. Il agit comme un pont entre deux tables, établissant une relation en référençant la clé primaire d'une autre table. Cela signifie que la clé étrangère assure l'intégrité des données en vous permettant de lier des données connexes, en assurant que les valeurs dans une table correspondent aux valeurs dans une autre. L'emploi de clés étrangères aide efficacement à maintenir une base de données structurée et organisée.

InnoDB : InnoDB est un type de table d'une base MySQL, le précédent type de table utilisé jusqu'alors était MyISAM. Le principal atout de l'utilisation de cette nouvelle technologie est la gestion des transactions lors de l'accès aux données. MySQL permet donc désormais "commit" et "roll-back", et devient donc un SGBD transactionnel, concrétisant ainsi le souhait d'une grande majorité des utilisateurs du produit.

Suppression en cascade : Une suppression en cascade (*cascade delete*) est une règle d'intégrité référentielle qui permet de supprimer automatiquement tous les enregistrements dépendants (enfants) d'une table liée lorsqu'on supprime l'enregistrement principal (parent)

Hachage : L'utilisation d'une fonction de hachage permet de ne pas stocker les mots de passe en clair dans la base mais uniquement de stocker une empreinte de ces derniers. Il est important d'utiliser un algorithme public réputé fort afin de calculer les dites empreintes.

BCrypt : Algorithme de hachage de mot de passe fort qui est résistant aux attaques par force brute. Il est plus lent que d'autres algorithmes, tels que SHA-256, ce qui rend plus difficile pour les attaquants de deviner les mots de passe.

2FA : L'authentification à deux facteurs (2FA) est un moyen de vérifier l'identité d'un utilisateur en demandant exactement deux éléments de preuve, tels que le mot de passe d'un compte en ligne (le premier facteur) et un code d'accès à usage unique d'une application d'authentification (le deuxième facteur).

TOTP : Un TOTP est un type de mot de passe à usage unique (OTP) qui génère des codes temporaires en utilisant le temps comme élément clé. Ces codes changent toutes les 30 à 60 secondes, ce qui les rend extrêmement difficiles à compromettre pour les cybercriminels. Dans le cas peu probable où ils découvriraient le code, sa courte durée de vie le rendrait rapidement presque inutile pour un attaquant.

VII) Source, annexes et bibliographie :

1 - Sources définitions :

<https://www.microsoft.com/fr-fr/security/business/security-101/what-is-devsecops#:~:text=D%20evSecOps%20est%20une%20approche%20du%20cycle%20de%20vie%20DevOps>.

<https://azure.microsoft.com/fr-fr/resources/cloud-computing-dictionary/what-is-devops#:~:text=D%C3%A9finition%20de%20DevOps&text=DevOps%20permet%20la%20coordination%20et%20plus%20performants%20et%20plus%20fiables>.

[https://www.cloudflare.com/fr-fr/learning/access-management/what-is-ssh/#:~:text=Le%20protocole%20Secure%20Shell%20\(SSH\)%20est%20une%20m%C3%A9thode%20permettant%20d%20les%20connexions%20entre%20les%20appareils](https://www.cloudflare.com/fr-fr/learning/access-management/what-is-ssh/#:~:text=Le%20protocole%20Secure%20Shell%20(SSH)%20est%20une%20m%C3%A9thode%20permettant%20d%20les%20connexions%20entre%20les%20appareils).

<https://www.elastic.co/fr/what-is/log-files>

<https://www.cloudflare.com/fr-fr/learning/access-management/token-based-authentication/>

<https://www.redhat.com/fr/topics/api/what-is-a-rest-api>

<https://www.lenovo.com/fr/fr/glossary/plugin/?orgRef=https%253A%252F%252Fwww.google.com%252F&srsltid=AfmBOoqGrUabIsYmkqITkElEDj6gXZxq7GylwHT5yZcpeNSGw0pLrl-l>

<https://www.oracle.com/fr/database/systeme-gestion-base-de-donnees-sgbd-definition/>

<https://www.ovhcloud.com/fr/learn/what-is-sql/>

<https://www.ovhcloud.com/fr/learn/what-is-nosql/>

<https://www.ibm.com/fr-fr/think/topics/primary-key>

https://www.lenovo.com/ca/fr/glossary/foreign-key/?srsltid=AfmBOoqXJf2KUMFWdkkzlAR_JoEOJo947eFXFVT1On6T6pfLSpc5MI75

<http://igm.univ-mlv.fr/~dr/XPOSE2002/MySQL/presentation2.htm>

<https://www.datacamp.com/fr/tutorial/sql-on-delete-cascade>

<https://introbd.dinf.ca/semaine6/cascade/>

<https://www.cnil.fr/fr/definition/hachage>

<https://academy.visiplus.com/ressources/definition/bcrypt>

<https://www.ibm.com/fr-fr/think/topics/2fa>

<https://proton.me/fr/blog/totp>

2 - Sources informations techniques :

Fail2Ban

<https://doc.ubuntu-fr.org/fail2ban>

<https://github.com/fail2ban/fail2ban>

Permissions Ubuntu

<https://doc.ubuntu-fr.org/permissions>

Droits des utilisateurs Ubuntu

<https://doc.ubuntu-fr.org/droits>

Cacti

<https://www.cacti.net/support/manuals>

<https://docs.cacti.net/>

Icinga

<https://icinga.com/docs/>

<https://icinga.com/docs/icinga-2/latest/doc/01-about/>

Docker :

<https://docs.docker.com/>

MySQL :

<https://dev.mysql.com/doc/>

MongoDB :

<https://www.mongodb.com/docs/>

3 - Annexes :

Lien vers le dépôt Github du projet de stage :

https://github.com/Sparrow0245/Projet_Stage_DEVOX

Sentinelle : documentation des versions

Présentation générale

Sentinelle est une solution de monitoring et de sécurisation pour serveur Ubuntu, conçue et développée dans le cadre du stage DevSecOps effectué au sein de l'entreprise DEVOX. L'objectif principal de cet outil est de proposer une supervision et une sécurisation fonctionnant entièrement en local, sans dépendance à des outils tiers, afin de limiter la surface d'exposition aux cyberattaques.

Le projet a été développé de manière itérative, en quatre versions successives. Ce document présente, pour chacune d'entre elles, le principe général et les apports par rapport à la version précédente.

Sentinelle version 1 : la solution manuelle

Cette première version se compose d'une dizaine de scripts qui se lancent à la main selon un certain ordre :

- lancement du script d'installation pour s'assurer que tout est à jour
- lancement du script de scan
- lancement du script adapté à la sortie retournée par le script de scan

Le principal avantage de cette solution est que l'administrateur peut superviser le scan et lui-même gérer les informations. Mais cette solution est chronophage et peut laisser passer des informations essentielles. Il est donc nécessaire de faire en sorte que la solution puisse tourner d'elle-même en arrière-plan sans intervention.

Sentinelle version 2 : la solution en arrière-plan

Cette version s'inscrit dans la continuité de la version 1. Cette fois-ci, il suffit de lancer le script d'installation et toute la solution s'installe, se déploie et tourne en arrière-plan sur le principe du CI/CD. Cela permet de ne pas avoir à lancer les scripts manuellement et de pouvoir inscrire toutes les informations remontées dans une base de données gérée par le SGBD MySQL.

Afin de davantage prendre en compte l'aspect cybersécurité, certains ajouts ont été faits pour gérer les attaques en bloquant les adresses IP agressives par un système de prison (ou jail), sur le modèle de Fail2Ban : si une adresse IP agit de manière suspecte ou tente des connexions répétées (via SSH par exemple, ou en cas d'attaque DDoS), des scripts isolent cette adresse IP aggressive et la placent sur une liste noire, l'empêchant ainsi de tenter toute connexion à l'avenir.

Cette version peut néanmoins être améliorée sur un point : la visualisation des informations. En effet, il est préférable d'avoir la possibilité de traiter visuellement les informations pour une bonne supervision.

Sentinelle version 3 : le dashboard simple

Afin de simplifier et d'améliorer l'utilisation de cet outil, la création d'un dashboard constitue une amélioration logique. Pour cela, il est nécessaire d'ajouter quelques scripts qui remontent des informations supplémentaires en temps réel sur l'état du serveur, comme la consommation de RAM, la consommation de disque, et la consommation de CPU. Le tout s'affiche sous forme de courbes dans une page web.

Cette première version du dashboard permet d'avoir une vision globale sur les évènements se déroulant sur le serveur en direct. Afin de compléter cette avancée, il peut être utile d'ajouter certains éléments de sécurité comme la remontée des alertes, ce qui permet de passer sur une version plus complète de l'application de monitoring en y ajoutant des éléments de cybersécurité.

Sentinelle version 4 : le dashboard complet

Pour cette version 4, le projet entre dans sa phase finale en se rapprochant d'une vraie application de monitoring. En plus de remonter les informations sur la consommation de disque, de RAM et de CPU, cette version introduit une vraie part de cybersécurité : désormais, le dashboard affiche toutes les notifications de sécurité.

Une nouveauté s'ajoute avec cette version : le dashboard administrateur. Cette page permet de visualiser, en plus des informations publiques :

- une page avec un profil administrateur plus complet avec authentification par **Token JWT**
- le journal détaillé des dernières événements et alertes
- les 50 derniers enregistrements de métriques
- l'espace SWAP
- une page d'historique avec toutes les informations récoltées

Cette version plus complexe possède plusieurs pages de visualisation, permettant ainsi de s'authentifier en profil administrateur, ou de traiter un type d'information en particulier.

Synthèse

Version	Fonctionnement	Apport principal
V1	Manuelle	Scripts Bash lancés à la main
V2	Arrière-plan	Automatisation Systemd + stockage MySQL + blocage IP (Fail2Ban)
V3	Dashboard simple	Visualisation temps réel CPU/RAM/Disque
V4	Dashboard complet	Authentification JWT + 2FA, journal des événements, espace SWAP, historique

Documentation technique : base de données Sentinelle

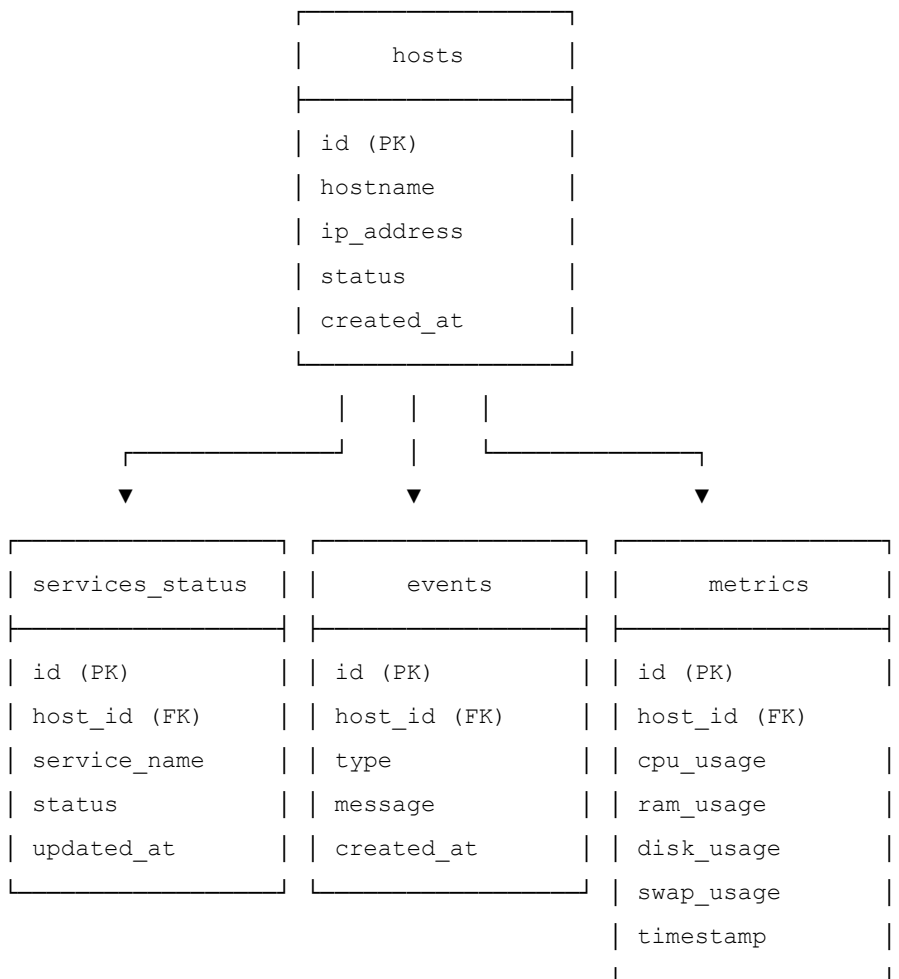
1. Présentation générale

La base de données de l'application Sentinelle, nommée **sentinelle**, a été conçue selon une structure relationnelle normalisée afin de minimiser la répétition d'informations et d'optimiser les performances de consultation du tableau de bord.

L'organisation repose sur cinq tables principales : **hosts**, **metrics**, **services_status**, **events** et **users**.

2. Schéma relationnel

La table **hosts** se trouve au centre de la structure. Les tables **metrics**, **services_status** et **events** y sont reliées par une relation de 1 à N via la clé étrangère **host_id**. De son côté, la table **users** demeure indépendante, ce qui permet d'isoler la couche d'authentification des données applicatives de surveillance.



users	(table indépendante)
id (PK)	
username	
password_hash	
totp_secret	
role	
created_at	

3. Description détaillée des tables

La table `hosts`

Cette table constitue le cœur de la base. Elle recense chaque machine surveillée.

Champ	Type	Description
id	INT (PK)	Identifiant unique de l'hôte
hostname	VARCHAR(255)	Nom d'hôte de la machine surveillée
ip_address	VARCHAR(45)	Adresse IP de l'hôte
status	ENUM	Statut général : <code>online</code> , <code>offline</code> , <code>warning</code>
created_at	TIMESTAMP	Horodatage de création de l'enregistrement

La table `metrics`

Cette table enregistre la consommation des ressources matérielles des hôtes.

Champ	Type	Description
id	BIGINT (PK)	Clé primaire dimensionnée en BIGINT pour absorber une importante volumétrie de relevés sur le long terme
host_id	INT (FK)	Référence vers <code>hosts.id</code>
cpu_usage	DECIMAL(5,2)	Taux d'utilisation du CPU, en pourcentage
ram_usage	DECIMAL(5,2)	Taux d'utilisation de la RAM, en pourcentage
disk_usage	DECIMAL(5,2)	Taux d'utilisation du disque, en pourcentage
swap_usage	DECIMAL(5,2)	Taux d'utilisation de l'espace SWAP, en pourcentage
timestamp	TIMESTAMP	Horodatage du relevé

Le type **DECIMAL(5,2)** est retenu pour l'ensemble des indicateurs de consommation afin de préserver la précision des pourcentages.

La table `services_status`

Cette table suit l'état des services système supervisés.

Champ	Type	Description
id	INT (PK)	Identifiant unique de l'enregistrement
host_id	INT (FK)	Référence vers <code>hosts.id</code>
service_name	VARCHAR(100)	Nom du service supervisé
status	ENUM	État du service : <code>active</code> , <code>inactive</code> , <code>failed</code>
updated_at	TIMESTAMP	Date de dernière mise à jour

La table `events`

Cette table sert de journal d'audit pour les alertes générées par le système.

Champ	Type	Description
id	BIGINT (PK)	Identifiant unique de l'événement
host_id	INT (FK)	Référence vers <code>hosts.id</code>
type	ENUM	Niveau de sévérité : <code>INFO</code> , <code>WARNING</code> , <code>CRITICAL</code>
message	TEXT	Message détaillé de l'événement
created_at	TIMESTAMP	Date de création de l'événement

La table `users`

Cette table gère les accès à l'interface web de supervision.

Champ	Type	Description
<code>id</code>	INT (PK)	Identifiant unique de l'utilisateur
<code>username</code>	VARCHAR(50)	Nom d'utilisateur, unique
<code>password_hash</code>	VARCHAR(255)	Empreinte du mot de passe, hachée via BCrypt
<code>totp_secret</code>	VARCHAR(64)	Clé secrète pour la double authentification (2FA/TOTP)
<code>role</code>	ENUM	Rôle d'accès : <code>admin</code> ou <code>viewer</code>
<code>created_at</code>	TIMESTAMP	Date de création du compte

4. Justification du choix du SGBD

Deux options s'offraient pour la gestion des données : MySQL (relationnel) ou MongoDB (NoSQL orienté document).

- **MongoDB** apporte une grande souplesse pour stocker des données non structurées sous forme de documents BSON/JSON sans imposer de schéma rigide. Cependant, cette flexibilité implique de devoir gérer la cohérence et l'intégrité des liaisons de données au niveau applicatif.
- **MySQL** est un SGBD relationnel classique et éprouvé, garantissant le respect strict des propriétés ACID (Atomicité, Cohérence, Isolation, Durabilité). Il s'appuie sur une organisation rigoureuse en tables et gère nativement l'intégrité référentielle par le biais de clés étrangères.

Le choix s'est naturellement porté sur MySQL. En effet, l'ensemble des données collectées par l'application (hôtes, métriques système, état des services, journaux d'événements et identifiants utilisateurs) présente une structure clairement définie et intrinsèquement relationnelle.

5. Mesures de durcissement (DevSecOps)

Dans une démarche DevSecOps favorisant la sécurité dès la conception (*Security by Design*), plusieurs choix techniques et configurations de durcissement ont été appliqués sur la base de données :

- **Moteur InnoDB** : l'ensemble des tables utilise ce moteur de stockage, qui garantit la prise en charge native des transactions ACID et assure la cohérence des données grâce au support des contraintes d'intégrité référentielle.
- **Suppression en cascade** (`ON DELETE CASCADE`) : configurée sur toutes les clés étrangères `host_id`. En cas de retrait d'un hôte dans la table `hosts`, toutes les métriques, statuts de services et événements associés sont supprimés automatiquement, ce qui évite la présence de données orphelines.
- **Encodage utf8mb4** associé à la collation `utf8mb4_unicode_ci` : assure le support complet de l'ensemble des jeux de caractères tout en protégeant l'application contre les failles d'injection liées aux conversions d'encodage.

- **Hachage des mots de passe** : aucun mot de passe n'est stocké en clair. Le champ `password_hash` enregistre uniquement des empreintes chiffrées générées via l'algorithme fort BCrypt.
 - **Double authentication** : l'intégration du champ `totp_secret` permet de gérer le 2FA basé sur des jetons temporels à usage unique (TOTP).
 - **RBAC** (contrôle d'accès basé sur les rôles) : le champ `role` limite les privilèges d'administration aux seuls profils autorisés.
 - **Écoute restreinte** : le SGBD est configuré pour n'écouter que sur l'interface locale (`127.0.0.1`), interdisant toute exposition directe du service MySQL sur le réseau extérieur.
-

6. Synthèse des relations

Table source	Clé étrangère	Table cible	Cardinalité
metrics	host_id	hosts	N à 1
services_status	host_id	hosts	N à 1
events	host_id	hosts	N à 1
users	—	—	table indépendante

Choix de la méthode d'authentification :

Sources :

<https://developer.okta.com/blog/2022/02/08/cookies-vs-tokens> (<https://developer.okta.com/blog/2022/02/08/cookies-vs-tokens>)

<https://auth0.com/blog/cookies-tokens-jwt-the-aspnet-core-identity-dilemma/#Cookie-or-Token-based-Authentication->
(<https://auth0.com/blog/cookies-tokens-jwt-the-aspnet-core-identity-dilemma/#Cookie-or-Token-based-Authentication->)

<https://blog.logto.io/fr/token-based-authentication-vs-session-based-authentication> (<https://blog.logto.io/fr/token-based-authentication-vs-session-based-authentication>)

Contexte :

Le protocole HTTP étant *stateless* (sans état), chaque requête est indépendante et n'est pas stockée sur le serveur. Ce dernier ne retient donc aucune information des échanges précédents. Un inconvénient apparaît donc avec ce protocole :

l'utilisateur doit se réauthentifier à chaque requête, ce qui complique une utilisation en pratique. Pour palier à ce problème, il existe deux approches pour maintenir l'état d'authentification : les **cookies session** et les **token JWT**.

Choix n°1 : Cookies session

Principe de fonctionnement :

Cette méthode de connexion repose sur le serveur pour conserver l'état d'authentification. Le flux est le suivant :

1. L'utilisateur soumet ses identifiants.
2. Le serveur valide les identifiants et crée un enregistrement de session en base de données (contenant un identifiant aléatoire, l'ID utilisateur, les dates de création et d'expiration, etc.).
3. Le `SessionID` est renvoyé au client sous forme de **cookie**.
4. À chaque requête suivante, le navigateur envoie automatiquement ce cookie.
5. Le serveur interroge sa base de données pour valider le `SessionID` et autorise (ou non) l'accès.

Cela implique que les données d'authentification sont stockées à la fois côté client pour le `SessionID` dans le cookies, et côté serveur avec l'enregistrement de session complet en base

Avantages et Inconvénients :

Avantages

Avantage	Détail
Gestion automatique par le navigateur	L'envoi du cookie à chaque requête est géré nativement par le navigateur, sans code supplémentaire côté frontend.
Stockage léger côté client	Un cookie de session peut ne stocker qu'un identifiant de quelques octets (environ 6 Ko pour un simple user ID).
Révocation en temps réel	L'accès peut être immédiatement révoqué côté serveur en supprimant l'enregistrement de session (déconnexion forcée par un administrateur, détection de compromission, etc.).
Protection contre le XSS	L'attribut <code>HttpOnly</code> empêche JavaScript d'accéder au cookie, réduisant la surface d'attaque XSS.
Partage entre sous-domaines	L'attribut <code>Domain</code> permet d'utiliser la même session sur <code>example.com</code> et ses sous-domaines.
Simplicité et fiabilité	La source de vérité est centralisée sur le serveur, ce qui rend les décisions d'autorisation directes et fiables.

Inconvénients

Inconvénient	Détail
Vulnérabilité CSRF	Les cookies sont envoyés automatiquement par le navigateur, y compris lors de requêtes provenant de sites tiers malveillants. L'attribut <code>SameSite</code> atténue ce risque mais peut dégrader l'expérience utilisateur.
Problèmes de scalabilité	Les sessions étant liées à un serveur, un déploiement en <i>load balancing</i> nécessite un magasin de sessions partagé (Redis, base de données centralisée), ce qui ajoute de la latence et de la complexité.
Consommation de ressources serveur	Chaque session active consomme de la mémoire ou de l'espace disque côté serveur. La performance peut se dégrader avec une grande base d'utilisateurs.
Inadapté aux API et applications mobiles	Les cookies sont difficiles à gérer sur les applications mobiles natives. Pour les API, qui sont par nature sans état, la gestion de sessions est une complexité superflue.
Risque de détournement de session	Le vol du cookie de session (<i>session hijacking</i>) peut permettre un accès non autorisé. L'attribut <code>Secure</code> (cookie uniquement sur HTTPS) est indispensable pour atténuer ce risque.

Choix n°2 : token JWT

Principe de fonctionnement :

Contrairement à la méthode de connexion précédente, dans l'authentification par token le serveur ne conserve aucun enregistrement de session. Toutes les informations nécessaires à la validation sont **encodées dans le token lui-même** et restent sur la machine de l'utilisateur.

Le flux est le suivant :

1. L'utilisateur soumet ses identifiants au serveur d'authentification.
2. Le serveur valide les identifiants et génère un JWT signé cryptographiquement.
3. Le JWT est renvoyé au client, qui le stocke (cookie, `localStorage`, `sessionStorage`).
4. À chaque requête suivante, le client envoie le JWT dans le header HTTP `Authorization: Bearer <token>`.
5. Le serveur vérifie la **signature** du JWT et ses **claims** (expiration, émetteur, audience) — sans aucun accès à une base de données.

Un token JWT est composée de 3 parties encodées en Base 64 et séparée par des points (.)

Avantages et Inconvénients :

Avantages

Avantage	Détail
Sans état et scalable	Le serveur n'a pas besoin d'interroger une base de données pour valider chaque requête. Particulièrement adapté aux architectures distribuées et aux microservices.
Multi-domaines et SSO	Un même JWT peut être accepté par plusieurs services ou domaines différents (<code>api.example.com</code> , <code>dashboard.example.com</code>), contrairement aux cookies qui sont liés à un domaine. Idéal pour le Single Sign-On (SSO).
Compatible mobile et API REST	Les tokens peuvent être stockés côté client et transmis facilement dans des headers HTTP, sans les contraintes des cookies sur les applications natives.
Informations embarquées	Le payload peut contenir des claims personnalisés (rôles, permissions) exploitables directement par le serveur destinataire, sans requête supplémentaire en base.
Vérification cryptographique	La signature garantit l'intégrité du token. Le contenu est visible mais non modifiable sans invalider la signature.

Inconvénients

Inconvénient	Détail
Révocation difficile	Un JWT ne peut pas être invalidé avant son expiration, sauf à implémenter une liste noire (<i>deny-list</i>), ce qui réintroduit un état côté serveur et contredit la nature stateless des JWT.
Données potentiellement périmées	Le payload représente un instantané au moment de l'émission. Si les droits d'un utilisateur changent (rétrogradation, suppression de compte), le JWT reste valide jusqu'à expiration.
Taille plus importante	Un JWT peut peser 300 octets ou plus, contre quelques octets pour un <code>SessionID</code> , ce qui alourdit chaque requête HTTP.

Inconvénient	Détail
Sécurité du stockage à la charge du développeur	Le stockage dans <code>localStorage</code> expose le token aux attaques XSS. Le stockage dans un cookie <code>HttpOnly</code> est recommandé, mais réduit l'avantage sur les cookies classiques.
Complexité de gestion	La rotation des clés de signature, la gestion des tokens de rafraîchissement (<i>refresh tokens</i>), et les stratégies de révocation nécessitent une implémentation rigoureuse.

Stratégies de révocation d'un JWT

La révocation d'un JWT est plus complexe que pour une session. Nous avons donc plusieurs approches possibles qui sont :

1. **Durée d'expiration courte (*exp*)** : Définir un claim *exp* très court (ex. 15 minutes) et utiliser un *refresh token* pour renouveler l'accès. Limite la fenêtre d'exploitation en cas de compromission.
2. **Liste noire (*deny-list*)** : Maintenir côté serveur une liste des tokens révoqués. Efficace mais réintroduit un état serveur et peut devenir volumineux.
3. **Point de révocation (*revocation endpoint*)** : Révoquer le *refresh token* sur le serveur d'autorisation. Les tokens d'accès ne seront plus renouvelés. Adapté aux flux OAuth 2.0.

Comparatif :

Critère	Cookies de session	Token JWT
Nature	Stateful (état côté serveur)	Stateless (état dans le token)
Stockage état	Base de données / cache serveur	Côté client (cookie, <code>localStorage</code> ...)
Révocation	Immédiate (suppression en base)	Difficile (nécessite liste noire ou expiration courte)
Scalabilité	Limitée sans stockage partagé	Excellente (pas de dépendance serveur)
Taille côté client	~6 Ko (simple SessionID)	~300 octets minimum
Multi-domaines / SSO	Non (lié à un domaine)	Oui
Applications mobiles	Difficile à gérer	Adapté nativement
APIs REST	Peu adapté	Très adapté
Protection XSS	Bonne (avec <code>HttpOnly</code>)	Risquée (si stocké dans <code>localStorage</code>)
Protection CSRF	Risque (mitigé par <code>SameSite</code>)	Absent (pas de cookie automatique)
Fraîcheur des données	Garantie (requête BDD à chaque fois)	Non garantie (snapshot au moment de l'émission)
Complexité d'implémentation	Simple	Modérée à élevée

Critère
Cas d'usage
principal

Cookies de session
Applications web traditionnelles, e-commerce, services bancaires

Token JWT
APIs, SPA, mobile, microservices, SSO